

Optimizing Coding-Agent Harnesses

Testing whether a runtime change helps on new tasks

CMSC 848N · W8-L2 · Thursday, October 22, 2026

What would convince us that an edit helps?

Identify a failure the runtime can change.

Specify the smallest intervention that tests the explanation.

Measure its effect on new tasks under comparable conditions.

Today: one concrete edit, the search around it, and the evidence needed to keep it.

The recorded test result can become stale

Illustrative trace: the successful test belongs to an earlier workspace state.

```
t=12  edit parser.py                # state A
t=13  pytest tests/test_parser.py # PASS on A
t=14  edit parser.py                # state B
t=15  submit()                     # B was not tested
```

Question: what must the harness remember at t=13?

Verification is tied to a workspace state

Proposed record format for the classroom experiment.

```
tested_state = digest(relevant_state) # before execution
result = execute(command)
record = {
    "workspace": tested_state,
    "command": command,
    "exit_code": result.exit_code,
    "output": digest(result.output),
}
# At submission, compare the current workspace digest.
```

Hash relevant files and environment/config identity. Detect changes during the test.

Freshness, test success, and correctness

The record links a command and its result to the state that was checked.

The result can still miss a bug or test the wrong behavior.

A mandatory gate can block valid work when tests are unavailable.

Submission requires an allowed check on the current state, or an explicit exception.

Harness optimization with fixed model weights

$$J(\theta, h) = \mathbb{E}_{x \sim \mathcal{D}, \tau \sim p_{\theta, h}(\cdot | x)} [R(x, \tau)]$$

$$h^* \in \arg \max_{h \in \mathcal{H}} J(\theta, h) \quad \text{subject to } C(\theta, h) \leq B$$

θ : model weights. h : prompts, tools, state handling, control flow.

x : task; τ : trajectory; R : task reward; C : deployment cost; B : budget.

Three optimization targets require different evidence

What is retained after the improvement step?

Target	Changed object	Evaluation question
Task-level search	A patch for this issue	Was the chosen patch correct?
Harness search	Runtime used on later issues	Does the change transfer?
Policy training	Model weights	Does the learned behavior transfer?

More successful attempts on familiar tasks do not establish a reusable improvement.

One round of harness search

A failure hypothesis motivates a small source change.

Run unit checks, then compare against the incumbent on selection tasks.

Retain the code, outcomes, cost, and reason for acceptance or rejection.

A useful record explains both what changed and why that change was retained.

Self-Harness proposes edits from recurring failures

The same fixed model executes tasks and proposes harness changes.

Trace evidence motivates bounded modifications to the runtime.

Acceptance uses pass counts on both the evidence split and a regression split.

Both splits influence selection. A final test needs another set of tasks.

Self-Harness's acceptance rule

$$\Delta_s = P_s(h') - P_s(h), \quad s \in \{\text{in}, \text{ho}\}$$

$$\Delta_{\text{in}} \geq 0, \quad \Delta_{\text{ho}} \geq 0, \quad \max(\Delta_{\text{in}}, \Delta_{\text{ho}}) > 0$$

P is the number of passed tasks on the indicated split.

Pass counts cannot decrease, but individual tasks can regress. Merged edits need rechecking.

A retained Self-Harness verification instruction

Adapted instruction from a retained coding edit in Self-Harness.

```
before_submit = (  
    "Check that the patch contains source changes. "  
    "Run a relevant local reproduction or test. "  
    "Inspect the output for failures. "  
    "Repair the source and rerun when needed."  
)
```

The model may follow a reminder. A gate enforces a specified submission condition.

Selecting the highest score selects some noise

$$\hat{J}_1 = J + \epsilon_1, \quad \hat{J}_2 = J + \epsilon_2, \quad \epsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$$

$$\mathbb{E}[\max(\hat{J}_1, \hat{J}_2) - J] = \frac{\sigma}{\sqrt{\pi}} > 0$$

Toy calculation: both candidates have exactly the same true quality.

The better measured score is optimistic even before adaptive search begins.

Deriving the expected selection bias

$$\max(\epsilon_1, \epsilon_2) = \frac{\epsilon_1 + \epsilon_2 + |\epsilon_1 - \epsilon_2|}{2}$$

$$\epsilon_1 - \epsilon_2 \sim \mathcal{N}(0, 2\sigma^2) \quad \Rightarrow \quad \mathbb{E} \max(\epsilon_1, \epsilon_2) = \frac{\sigma}{\sqrt{\pi}}$$

The mean-zero terms cancel. The expected absolute difference is positive.

An independent evaluation of the selected candidate avoids reusing this selected noise.

A bound for a fixed set of candidates

$$\Pr \left[\max_{k \leq K} (\hat{J}_k - J_k) > \varepsilon \right] \leq K e^{-2n\varepsilon^2}$$

$$\varepsilon = \sqrt{\frac{\log(K/\delta)}{2n}}$$

K candidates fixed before n i.i.d. evaluation tasks; rewards in $[0,1]$; failure probability δ .

Adaptive reuse of the same task feedback violates that fixed-candidate condition.

Development, selection, and final testing

A score used to make a decision is part of optimization.

Split	Permitted use	Later claim
Development	Inspect traces and propose edits	Mechanism diagnosis
Selection	Rank, reject, or stop candidates	Choose one harness
Final test	Evaluate frozen choice once	Estimate new-task performance

If final-test feedback triggers another edit, that data has entered development.

Task separation must reflect the transfer claim

A new issue in a familiar repository is one kind of transfer.

An unfamiliar repository or tool environment is a stronger change.

Related issues, public fixes, and shared task templates can connect splits.

Choose and report the split before looking at which partition gives the largest gain.

Baselines with matched feedback and compute

Keep the harness fixed and sample several candidate patches.

Keep the harness fixed and refine a patch using permitted feedback.

Search over the harness, then evaluate its frozen result.

Match feedback access and total compute, including retries and refinement.

Search cost and deployment cost

$$C_{\text{total}}(M) = C_{\text{search}} + M c_{\text{new}}$$

$$M > \frac{C_{\text{search}}}{c_{\text{base}} - c_{\text{new}}} \quad (c_{\text{new}} < c_{\text{base}})$$

M: future tasks; c: cost per task. Assume comparable quality and a stable task distribution.

If the new harness costs more per task, justify it through a declared quality–cost tradeoff.

Four conditions for testing the freshness gate

Proposed experiment. B and C receive identical records and reminders.

Arm	Runtime	Comparison
A	Current baseline	Reference
B	Record, reminder, and enforced gate	B – A: full intervention
C	Same record and reminder, no gate	B – C: enforcement
D	Baseline + matched extra budget	B – D: matched compute

A–D share the model, public feedback, task environments, and evaluation rules.

Unit tests for the freshness gate

Deterministic checks for the proposed implementation.

Case	Expected handling
Edit after a passing check	Invalidate the record
Untracked source changes	Relevant digest changes
Test runner cannot start	Infrastructure failure
No runnable test	Logged exception, never a pass

Also test timeouts, modified test files, and config/dependency changes.

Estimating a paired difference in task success

$$d_i = \frac{1}{r} \sum_{j=1}^r [R(h_{\text{new}}, x_i, j) - R(h_{\text{base}}, x_i, j)]$$

$$\hat{\Delta} = \frac{1}{n} \sum_{i=1}^n d_i$$

For enforcement, compare B with C on each task. Use r runs on each of n tasks.

Report cost and uncertainty. Resample tasks, or repository clusters when appropriate.

Measurements that test the proposed mechanism

Did stale submissions become less frequent?

How many valid repairs were blocked or delayed?

Did test selection or environment failures change?

Repeated rollouts assess a frozen harness. Repeated searches assess the optimizer.

RRSI constrains proposals and candidate selection

Limit how many independent edits one proposal can bundle.

Screen task-specific leakage and retain a full edit history.

Check score, resource growth, and domain-specific guardrails before acceptance.

These rules constrain discrete source edits and resource growth. Model weights remain fixed.

RRSI's candidate filter

Source excerpt: d starts rejected. Final acceptance and logging are omitted.

```
floor = S_star - delta
if ev.S < floor:
    return d
ok, why = cost_rule(dS, dC, nov, delta, cfg)
if not ok:
    return d
if guards:
    return d
```

S_star: best retained score. delta: noise allowance. A nonempty guards list means a violation.

Two branches of RRSI's cost rule

$$\Delta C = \frac{C' - C}{C}, \quad \Delta S = S' - S$$

$$\begin{cases} \Delta C \leq \beta_0 + \beta_1 \Delta S, & \Delta S > \delta \\ w_s \Delta S - w_c \Delta C + w_n \nu > 0, & \Delta S \leq \delta \end{cases}$$

ν counts structural component types never used in a winning edit.

The score floor and domain guards still apply in both cases.

RRSI's evolution and transfer results

Reported coding results with the same frozen policy.

Benchmark	Role	Base → RRSI
Terminal-Bench 2.1	Used for evolution	74.2 → 80.2
SWE-bench Verified	Transfer evaluation	82.0 → 83.8

Pass rates (%), Claude Opus 4.8. These populations have different task difficulty.

Component effects depend on model and budget

A richer action interface can help a model that struggles with shell commands.

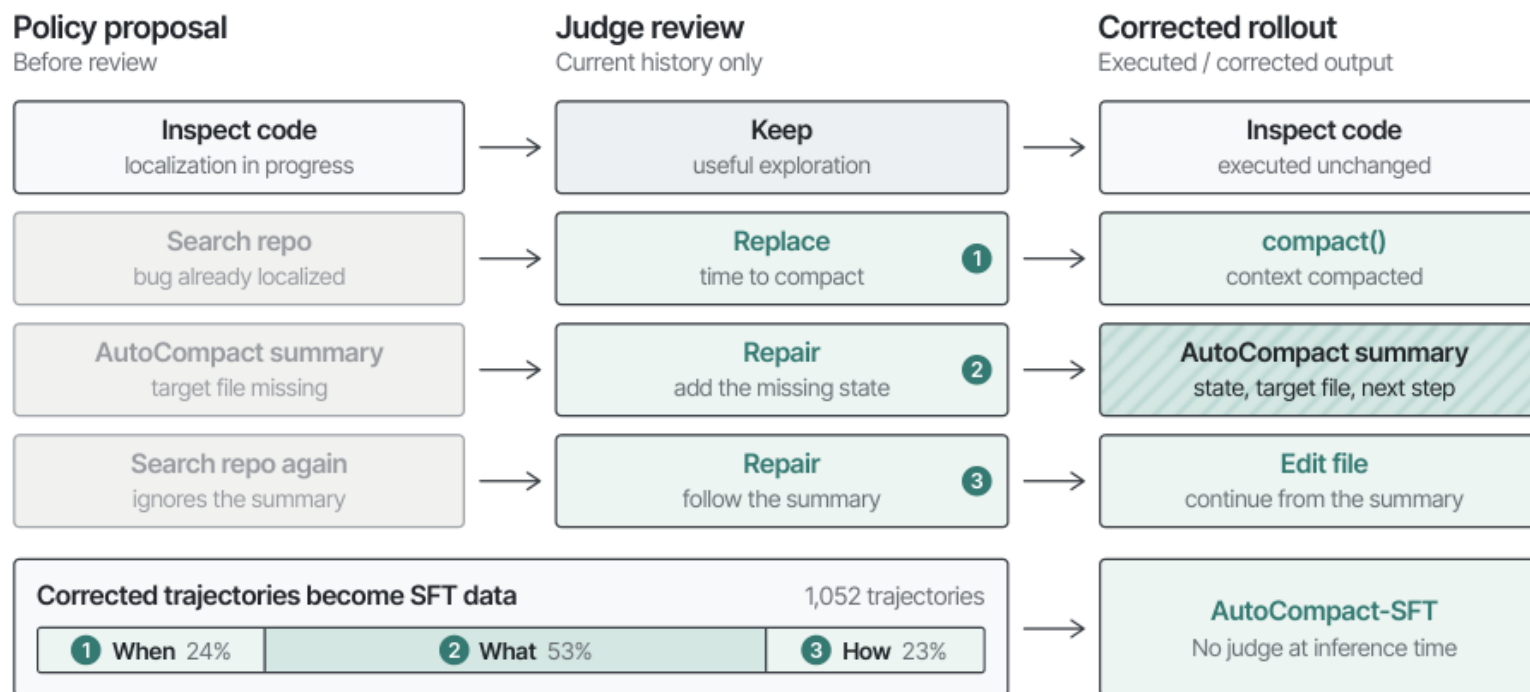
Context management can matter most when the available window is tight.

This study tests a persistent planning scaffold with instructions and a plan tool.

Treat model × component × budget as an interaction to test.

AutoCompact trains decisions about compaction

The judge corrects a decision before the environment executes it.



Corrected rollouts train SFT, then outcome-based RL. No judge runs at deployment.

Compaction changes the policy's next context

$$z_t = g_\theta(H_t), \quad a_{t+1} \sim \pi_\theta(\cdot \mid x, z_t, o_{t+1})$$

$$\theta^* \in \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta, h}} [R(x, \tau)]$$

H_t : history; z_t : summary; x : task; o_{t+1} : next observation. The summary can omit needed evidence.

Omitting a failed test or a workspace change can alter the next action.

Exercise: testing the freshness gate

Hypothesis: freshness enforcement improves repairs by preventing stale submissions.

Choose one alternative explanation and a control that distinguishes it.

Specify a result that would make you remove or revise the gate.

Four minutes in pairs. Keep the model, allowed feedback, and budget explicit.

Interpreting the experiment's possible outcomes

Illustrative outcomes for discussion. These are not measured results.

Observation	Possible interpretation
Fewer stale patches; same success	Freshness improves. Utility unclear.
Higher success and cost	Compare at equal budget
Gain only on selection tasks	Test for narrow adaptation
Gate blocks valid repairs	Review logic and exception policy

Mechanism evidence, utility, and transfer are separate conclusions.

Reproducing the harness and its search procedure

Pin model, harness, environment, task partitions, and access policy.

Save proposed diffs, rejected candidates, evaluator outputs, and full cost accounting.

Freeze the selected artifact before running the final test.

Report rollout variability, task variability, and search variability separately.

Evidence for a reusable harness improvement

Does the runtime intervention change the intended behavior?

Does it help on new tasks at the deployment budget?

Is the improvement worth its added complexity and failure modes?

A strong claim identifies the mechanism, the evidence, and the boundary of transfer.

Readings and code for harness optimization

Primary sources and code for this lecture.

Source	Focus
Self-Harness, Algorithm 1	Information used in acceptance
RRSI + selection.py	Candidate retention rules
AutoCompact, §2	The object being trained
Evaluation studies	Controls needed for a transfer claim

Full version-pinned links and reuse credits are in the companion source guide.